Weekly Assignment 10:
Dynamic Programming 2

1. (15 points) Using the DP algorithm discussed during the lecture, determine a longest common subsequence of the strings $\langle 1, 0, 0, 1, 0, 1, 0, 1 \rangle$ and $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0 \rangle$. Show the matrix M with the trace back path and the longest common subsequence.

2. (25 points) You are given a string of n characters $s = s[1], ..., s[n]$, which you believe to be a corrupted text document in which all spaces and punctuation have vanished (so that it looks something like "itwasthebestoftimes..."). You wish to reconstruct the sequence of words in the document using a dictionary, which is given to you as a Boolean function $d(w)$ that takes a string w as input and returns `true` if w is a valid word and `false` otherwise. Give a dynamic programming algorithm that determines whether the string s can be reconstituted as a sequence of valid words. Describe the recurrence equations on which your algorithm is based, explain why these equations hold, and analyze the time complexity of your algorithm. You may assume that calls to $d(w)$ take constant time.

3. (30 points) Imagine you place a knight chess piece on a phone dial pad. This chess piece moves in an uppercase "L" shape: two steps horizontally followed by one vertically, or one step horizontally then two vertically:

   Suppose you dial keys on the keypad using only hops a knight can make. Every time the knight lands on a key, we dial that key and make another hop. The starting position counts as being dialed. Give an efficient dynamic programming algorithm for the following problem: How many distinct numbers can you dial in N hops from a particular starting position. For example, with $N = 2$ and starting from position 1, we may dial the numbers 183, 181, 161, 167 and 160. Specify the recursion equations on which your algorithm is based.

4. (30 points) A *pattern* `p` is a string which possibly contains some occurrences of the symbols `*` and `?`. For an arbitrary string `w`, we say `w` *matches* the pattern `p` if it is possible to obtain `w` from `p` by replacing each `*` by a (possibly empty) string, and each `?` by a single character. For instance:

- `abbb` matches `a*b`
- `abbb` does not match `a?b` (since `?` must be replaced by a single character)
- `abbb` matches `a?b?`
- `abbaba` matches `a*b*a`
- `abbabb` does not match `a*b*a`.

A string `w` can match a pattern `p` in multiple ways; for instance, we can get `abbaba` from `a*b*a` in three ways (replacing the first star by the empty string and the second by `bab`; or replacing the first by `b` and the second by `ab`; or replacing the first by `bba` and the second by the empty string). Similarly, `abbb` matches `a*b*` in three ways, but `abbb` matches `a*b` in only one way.

Describe an algorithm which takes a string `w` and a pattern `p` and returns the number of ways in which `w` matches `p`. We assume for simplicity that `w` does not contain `*` or `?`. Explain your answer, and analyse correctness and complexity of your solution.